

Plan van Aanpak

- Blockytest -

"Low-code testautomatisering via visuele blokken"



Dragonslayers

Project: BlockyTest

Team: *Dragon Slayers*

Opleiding: HBO-ICT – Business IT Management

Onderwijsinstelling: Hogeschool van Amsterdam

Opdrachtgever:

Rachid Kherrazi

Docenten:

Philip Lafeber

Jelle Kaptein

Teamleden:

Ryan James Tiangco

Amira El Hafidi

Noura Ezzafzafi

Adel Al-Rammah

Contact:

noura.ezzafzafi@hva.nl

amira.el.hafidi@hva.nl

ryan.james.tiangco@hva.nl

adel.al.rammah@hva.nl

Datum: 19 maart 2026

Inhoud

1. Probleemstelling.....	4
1.1 Situatieschets	4
1.2 Aanleiding	4
1.3 Probleemanalyse	4
2. Doelstelling	4
2.1 Projectdoel.....	5
2.2 Koppeling met Sustainable Development Goals (SDG).....	5
3. Opdrachtbeschrijving	6
3.1 Het eindresultaat (MVP)	6
3.2 Projectorganisatie	6
3.3 Tussenproducten	6
3.5 AS-IS versus TO-BE Proces.....	7
4. Risicopreventie	8
4.1 Risicoanalyse methode: ISO 31000.....	8
4.2 Risicopreventie	8
5. Werkwijze	9
5.1 Scrum-methodiek.....	9
5.2 Betrokkenen.....	9
5.3 Middelen	10
6. Technisch Kader	10
6.1 Bedrijfsprocessen (BPMN)	10
6.2 Gegevensstructuur (ERD)	11
6.3 Use Case Diagram	11
6.4 High-fi schetsen.....	12
6.5 Projectgrenzen (Scope)	13
7. Kwaliteit en Begroting	14
7.1 Definition of Done (DoD).....	14
7.2 Story Points & Weging.....	14
8. Planning	15
8.1 De Sprintcyclus	15
8.2 De Roadmap (Sprintdoelen).....	16
9. Ondertekening.....	16

1. Probleemstelling

1.1 Situatieschets

Bij Wailsalutem BV is testautomatisering cruciaal voor de softwarekwaliteit, maar de huidige tools vormen een drempel: ze vereisen diepe programmeerkennis. Hierdoor ontstaat een kloof tussen de mensen die de scenario's bedenken (de business) en de mensen die ze technisch kunnen uitvoeren (de developers).

1.2 Aanleiding

Opdrachtgever Rachid Kherrazi wil deze kloof dichten. Het doel is een webtool genaamd BlocklyTest, waarin gebruikers testflows bouwen met visuele blokken in plaats van code. Dit project richt zich op de ontwikkeling van een *Minimum Viable Product* (MVP) om te bewijzen dat testautomatisering ook zonder programmeerkennis toegankelijk kan zijn.

1.3 Probleemanalyse

Momenteel kunnen alleen medewerkers met programmeerkennis testautomatisering uitvoeren. Niet-technische medewerkers kunnen wel testscenario's bedenken, maar niet zelfstandig omzetten naar geautomatiseerde tests.

Hierdoor ontstaan verschillende knelpunten:

- **Hoge afhankelijkheid:** Testers moeten altijd wachten op de beschikbaarheid van developers.
- **Trage doorlooptijd:** Het testproces duurt onnodig lang door technische vertragingen.
- **Communicatieruis:** Er is veel extra afstemming nodig om de business-wensen te vertalen naar code.

2. Doelstelling

Door een visuele laag over complexe code heen te leggen, veranderen we de manier waarop teams samenwerken aan softwarekwaliteit. Technologie wordt hiermee een hulpmiddel voor het hele team, niet alleen voor de programmeurs.

2.1 Projectdoel

Wij bouwen een low-code webapplicatie waarin gebruikers blokken naar een werkveld slepen om testflows te vormen. De tool vertaalt deze visuele handelingen direct naar werkende testcode. Dit levert de organisatie direct resultaat op:

- Snelheid: Tests kunnen direct door testers worden gebouwd, zonder te wachten op een developer.
- Focus: Developers kunnen zich concentreren op complexe softwarebouw in plaats van op eenvoudige testscripts.
- Kwaliteit: Door de visuele interface worden er minder fouten gemaakt in de logica van de tests.

2.2 Koppeling met Sustainable Development Goals (SDG)

Het project sluit aan bij meerdere Sustainable Development Goals (SDG's) van de United Nations (United Nations Development Programme [UNDP], z.d.). De invulling van deze doelen voor de Nederlandse context is gebaseerd op de kaders van het Ministerie van Algemene Zaken (2026).

SDG 3 – Goede gezondheid en welzijn

Testautomatisering kan bijdragen aan betrouwbaardere software. Wanneer software beter getest wordt, neemt de kans op fouten af.

Betrouwbare software is belangrijk in sectoren zoals de zorg, waar systemen vaak gebruikt worden voor patiëntgegevens en zorgprocessen. Door testen toegankelijker te maken kunnen meer medewerkers bijdragen aan het controleren van softwarekwaliteit

SDG 9 – Industrie, innovatie en infrastructuur

BlocklyTest maakt gebruik van visuele programmeerblokken en sluit daarmee aan bij moderne ontwikkelmethoden zoals low-code en visuele programmeeromgevingen.

Deze aanpak ondersteunt digitale innovatie binnen organisaties en maakt softwareontwikkeling toegankelijker voor een bredere groep gebruikers.

SDG 10 – Ongelijkheid verminderen

Binnen veel organisaties bestaat een verschil tussen technische en niet-technische medewerkers. Door testautomatisering toegankelijker te maken via een visuele tool kunnen ook medewerkers zonder programmeerkennis bijdragen aan het testproces.

Hierdoor wordt de kloof tussen business en IT kleiner.

3. Opdrachtbeschrijving

3.1 Het eindresultaat (MVP)

Het doel is een werkend prototype waarmee iedereen zonder programmeerkennis een test kan opzetten. De belangrijkste functies zijn:

- Visuele editor: Een sleep-en-neerzet interface in de browser.
- Testacties: Kant-en-klare blokken voor klikken, typen en controleren.
- Slimme vertaling: Blokken worden direct omgezet naar Robot Framework-code.
- Export: De gemaakte tests kunnen direct worden opgeslagen en gebruikt.

Het resultaat is een werkend prototype dat laat zien dat testautomatisering ook mogelijk is zonder programmeerkennis.

3.2 Projectorganisatie

Het team Dragon Slayers (Ryan, Amira, Adel en Noura) is verantwoordelijk voor de volledige bouw, van onderzoek tot de laatste demo.

Gedurende het project vindt regelmatig overleg plaats met de opdrachtgever. Tijdens deze momenten wordt voortgang, feedback en mogelijke verbeteringen besproken, zodat het eindproduct goed aansluit op de behoeften van de organisatie.

3.3 Tussenproducten

Tijdens het project worden verschillende tussenproducten opgeleverd die laten zien hoe het MVP stap voor stap ontstaat.

De belangrijkste tussenproducten zijn:

Tussenproduct	Beschrijving
Onderzoeksdocument	Resultaten van het onderzoek naar het gebruik van Blockly en Robot Framework binnen een webapplicatie.
Functioneel ontwerp	Beschrijving van de belangrijkste functionaliteiten van de applicatie en hoe gebruikers met de tool werken.
Interface ontwerp	Low-fi en later high-fi ontwerp van de gebruikersinterface van de BlocklyTest editor.
Eerste werkende editor	Een basisversie van de visuele editor waarin blokken geplaatst kunnen worden.

Codegeneratie prototype	Een versie waarin blokken automatisch worden vertaald naar testcode.
Werkende demo	Een demonstratie waarin een eenvoudige testflow (bijvoorbeeld een login-test) automatisch wordt uitgevoerd.
Eindproduct (MVP)	Het uiteindelijke werkende prototype inclusief documentatie en demonstratie.

Tabel 1

Overzicht van de op te leveren tussenproducten en hun beschrijving.

Bron: Eigen uitwerking (2026).

Door deze tussenproducten op te leveren kan het team regelmatig feedback ontvangen en het product waar nodig verbeteren.

3.5 AS-IS versus TO-BE Proces

Geautomatiseerd testen maakt het werk sneller en makkelijker. Je hebt minder hulp van developers nodig en tests kunnen direct uitgevoerd worden. Innowise Group bevestigt dit en legt uit dat automatisering zorgt voor snellere feedback en een efficiënter testproces (Innowise Group, z.d.).

Toch is er in de huidige situatie nog een duidelijke kloof tussen business en IT. Met BlocklyTest wordt deze kloof kleiner, omdat gebruikers zelf tests kunnen bouwen met visuele blokken.

Aspect	Huidige Situatie (AS-IS)	Nieuwe Situatie (TO-BE)
Proces	Business bedenkt scenario → Wacht op developer → Developer schrijft code.	Business sleept visuele blokken → Code wordt direct automatisch gegenereerd.
Afhankelijkheid	Testers zijn volledig afhankelijk van programmeerkennis van developers.	Testers kunnen zelfstandig geautomatiseerde tests opzetten en uitvoeren.
Snelheid	Lange doorlooptijd door communicatie en technische vertraging.	Korte doorlooptijd; tests kunnen direct na het bedenken worden gebouwd.

Tabel 2

Vergelijking tussen de huidige situatie (AS-IS) en de gewenste situatie met BlocklyTest (TO-BE).

Bron: Eigen uitwerking (2026).

4. Risicopreventie

Tijdens het uitvoeren van dit project kunnen verschillende risico's ontstaan die invloed hebben op de planning, samenwerking of kwaliteit van het eindresultaat. Daarom is het belangrijk om mogelijke risico's vooraf te identificeren en maatregelen te bepalen om deze te beperken.

4.1 Risicoanalyse methode: ISO 31000

Voor het risicomanagement maken wij gebruik van de internationale standaard ISO 31000. Deze standaard is toepasbaar op alle organisaties, ongeacht type, grootte, activiteiten of locatie, en heeft betrekking op alle soorten risico's (ISO, 2018).

Dit is binnen het bedrijfsleven de meest erkende methode voor het omgaan met risico's. (IB&P, 2020). Wij passen de vijf stappen van ISO 31000 (Identificatie, Analyse, Evaluatie, Behandeling en Monitoring) toe binnen onze Scrum-werkwijze:

- **Risico-identificatie:** Tijdens de *daily stand-up* ontdekt het team dagelijks nieuwe risico's.
- **Risicoanalyse:** Tijdens de *sprintplanning* kijken we naar de kans en impact van taken.
- **Risico-evaluatie:** Tijdens de *sprintreview* bekijken we met de opdrachtgever welke risico's het belangrijkst zijn.
- **Risicobehandeling:** In de *retrospective* nemen we maatregelen om procesrisico's te verminderen.
- **Monitoring:** Door sprints van drie weken worden risico's snel zichtbaar en kunnen we direct bijsturen.

4.2 Risicopreventie

1. Onvoldoende tijdsbesteding door andere opdrachten en verantwoordelijkheden Dit is een realistisch risico. De teamleden hebben te maken met **parallelle assignments, deadlines en persoonlijke verantwoordelijkheden**. Als de focus verslapt, leidt dit direct tot vertraging in de sprint en een lagere kwaliteit van het MVP.

Preventie: We maken een zeer gedetailleerde sprintplanning in GitLab. We spreken af dat achterstanden direct (binnen 24 uur) worden gemeld in de groeps-app, zodat taken herverdeeld kunnen worden voordat de deadline in gevaar komt.

2. Onvoldoende communicatie binnen het team Dit kan leiden tot misverstanden, dubbel werk of taken die niet worden uitgevoerd.

Preventie: We maken duidelijke afspraken over communicatie. Teamleden houden elkaar op de hoogte en bespreken problemen direct. GitLab dient als bron van waarheid voor de status van taken.

3. Problemen met GitLab en Versiebeheer Er bestaat een risico dat er fouten worden gemaakt bij het werken met branches, het pushen van code of het bijhouden van issues.

Preventie: Teamleden werken zorgvuldig volgens vaste afspraken. Issues worden voorzien van acceptatiecriteria en we controleren waar nodig elkaars werk (code reviews).

4. Ongelijke taakverdeling of gebrek aan kennis Verschillende niveaus van ervaring kunnen ervoor zorgen dat taken voor sommigen te moeilijk zijn, wat de voortgang remt.

Preventie: Bij het verdelen van taken houden we rekening met sterke punten. Wanneer iemand vastloopt, wordt dit direct besproken en helpen teamleden elkaar.

5. Planning en deadlines Taken die niet op tijd worden afgerond zorgen voor tijdsdruk aan het einde van de sprint.

Preventie: Het werk wordt opgedeeld in kleine, behapbare issues in GitLab. Tijdens de sprint controleren we regelmatig of we nog op schema liggen en passen de planning aan indien nodig.

5. Werkwijze

Om de plannen voor BlocklyTest soepel uit te voeren, werkt het team op een vaste manier samen. Duidelijke afspraken over rollen en hulpmiddelen zorgen ervoor dat iedereen weet wat er moet gebeuren en dat het prototype op tijd af is.

5.1 Scrum-methodiek

Het team werkt in periodes van drie weken (sprints). Elke periode begint met een planning en eindigt met een review aan de opdrachtgever. Tussendoor bespreekt het team dagelijks de voortgang en verbeteren we de onderlinge samenwerking.

5.2 Betrokkenen

- **Projectteam (Dragon Slayers):**
Verantwoordelijk voor onderzoek, ontwerp, realisatie en testen.
- **Opdrachtgever (Rachid Kherrazi - Wailsalutem BV):**
Levert input, stelt prioriteiten en geeft feedback.
- **Begeleider (HvA):**

Bewaakt de voortgang en het procesmatige niveau van het project.

5.3 Middelen

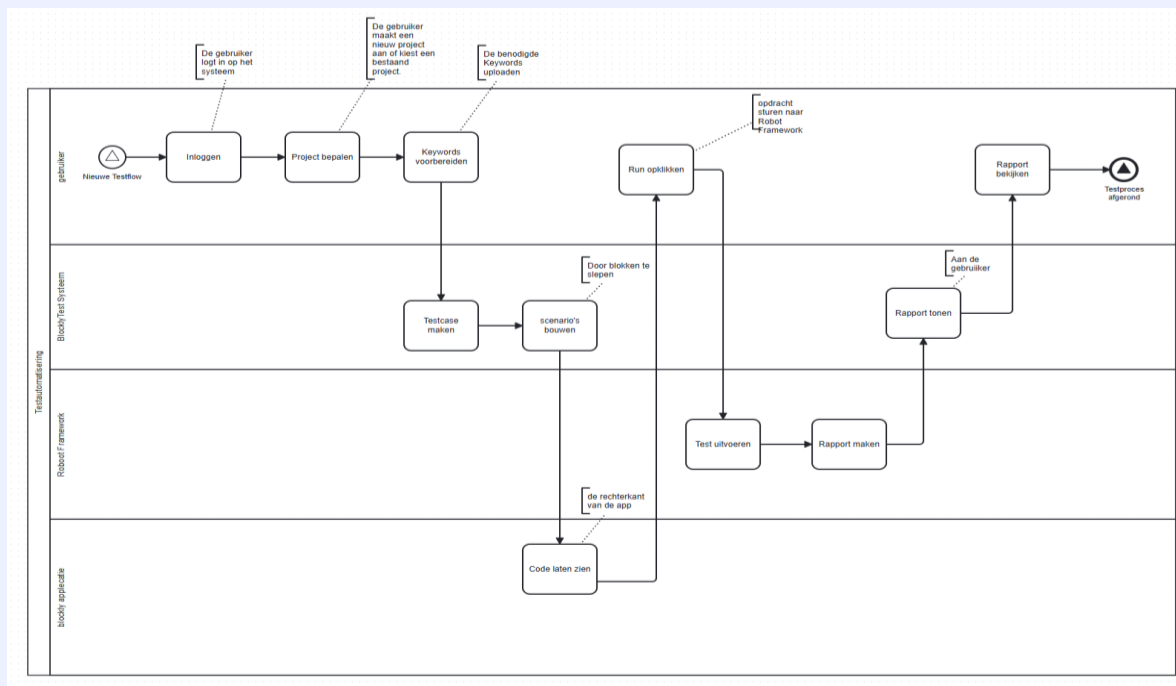
Er wordt gewerkt met open-source middelen: Google Blockly (visuele interface), Robot Framework (codegeneratie), GitLab (versiebeheer) en Microsoft Teams (communicatie).

6. Technisch Kader

Een goed idee werkt pas als de techniek erachter klopt. Door vooraf de grenzen van het systeem en de structuur van de data te bepalen, bouwen we een logische en stabiele oplossing voor de gebruiker.

6.1 Bedrijfsprocessen (BPMN)

Het BPMN-diagram (Figuur 1) visualiseert de flow: van de gebruiker die blokken sleept, naar de automatische codegeneratie, tot de uiteindelijke uitvoering van het testscript.



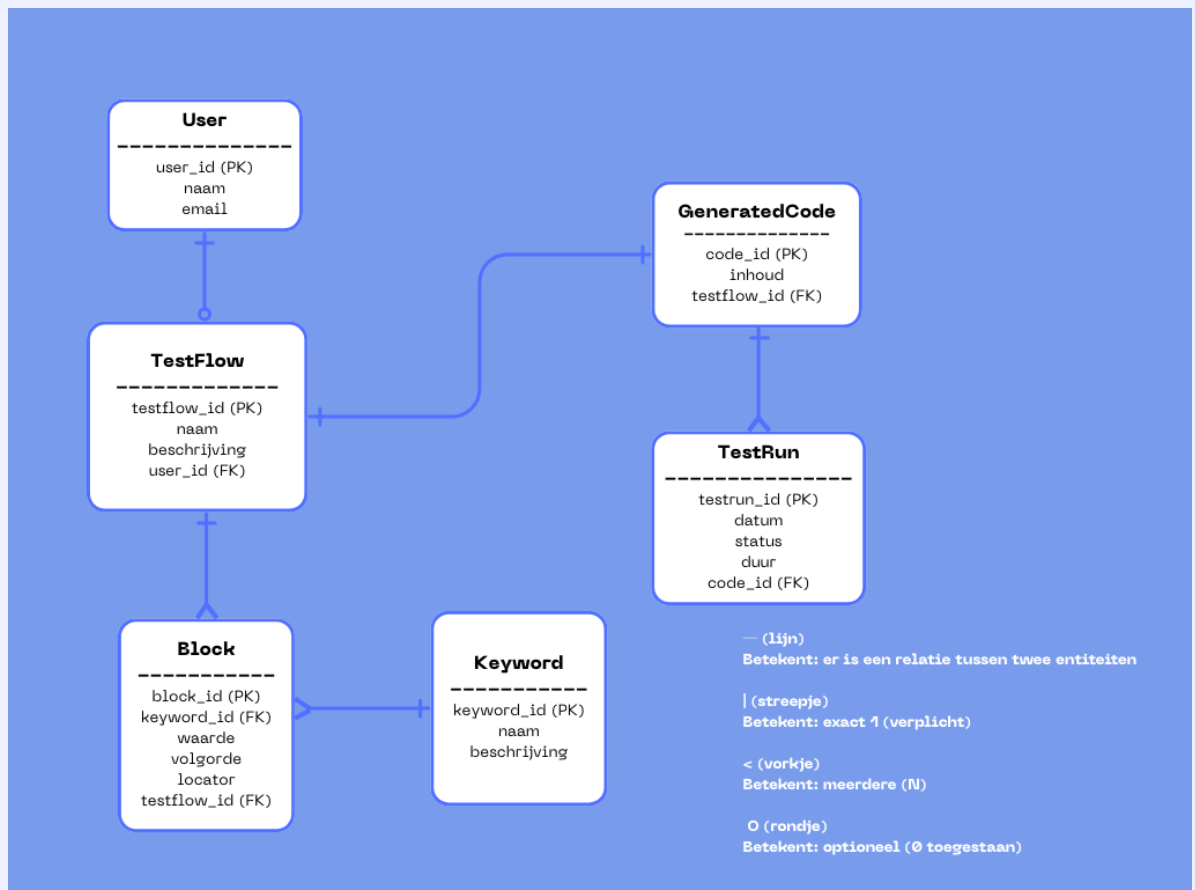
Figuur 1

BPMN-proces van het samenstellen en uitvoeren van een testflow in BlocklyTest.

Bron: gebaseerd op BPMN.org (z.d.).

6.2 Gegevensstructuur (ERD)

Het Entity-Relationship Diagram (Figuur 2) toont hoe data over gebruikers, testflows, visuele blokken en de gegenereerde code binnen de database aan elkaar gekoppeld zijn. Een ERD wordt volgens Lucidchart gebruikt om de structuur van een database visueel weer te geven en duidelijk te maken hoe entiteiten met elkaar in relatie staan (Lucidchart, z.d.).



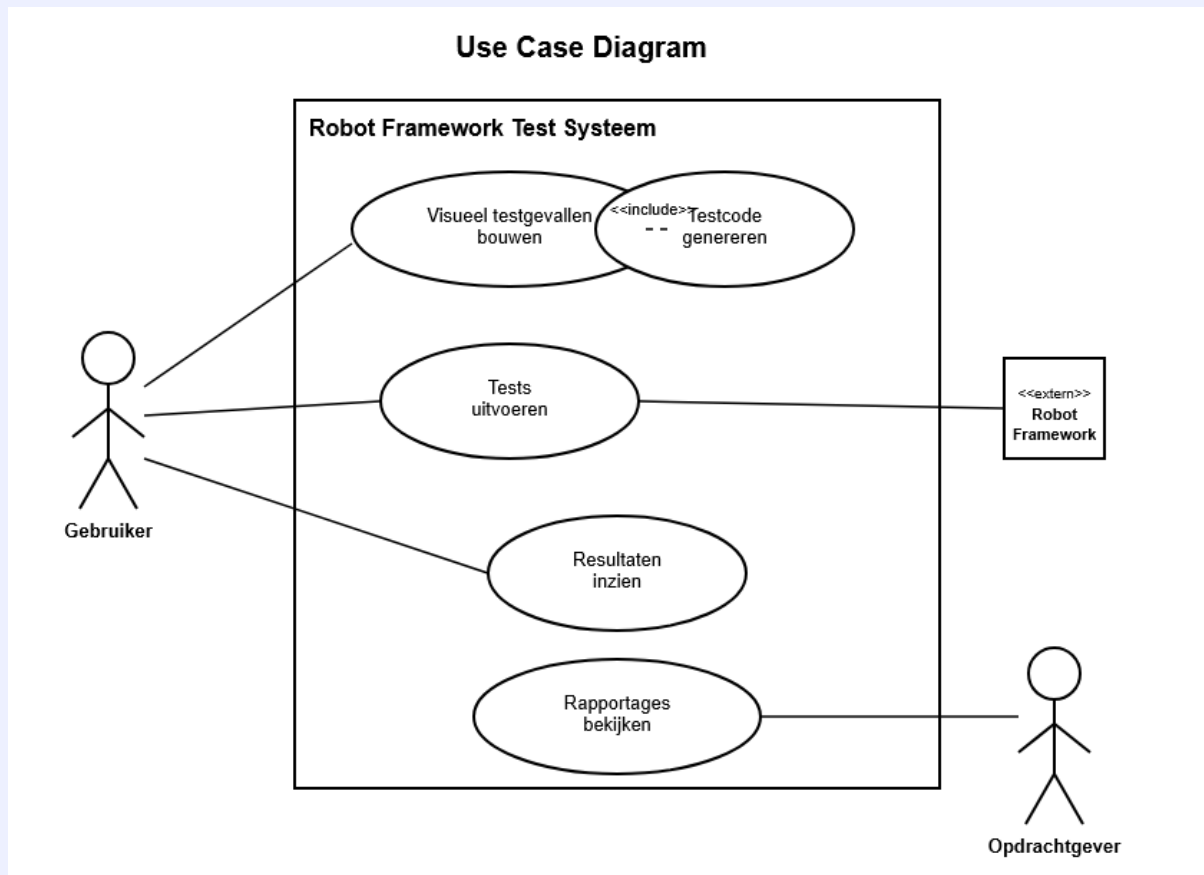
Figuur 2

Entity-Relationship Diagram (ERD) van de BlocklyTest database-structuur.

Bron: Eigen uitwerking (2026).

6.3 Use Case Diagram

Het Use Case Diagram (Figuur 3) laat zien hoe de gebruiker en de opdrachtgever interactie hebben met het systeem, van het visueel bouwen van testgevallen tot het bekijken van testresultaten uit het Robot Framework. Use case-diagrammen worden gebruikt om functionele interacties tussen actoren en het systeem inzichtelijk te maken (Object Management Group, 2017).



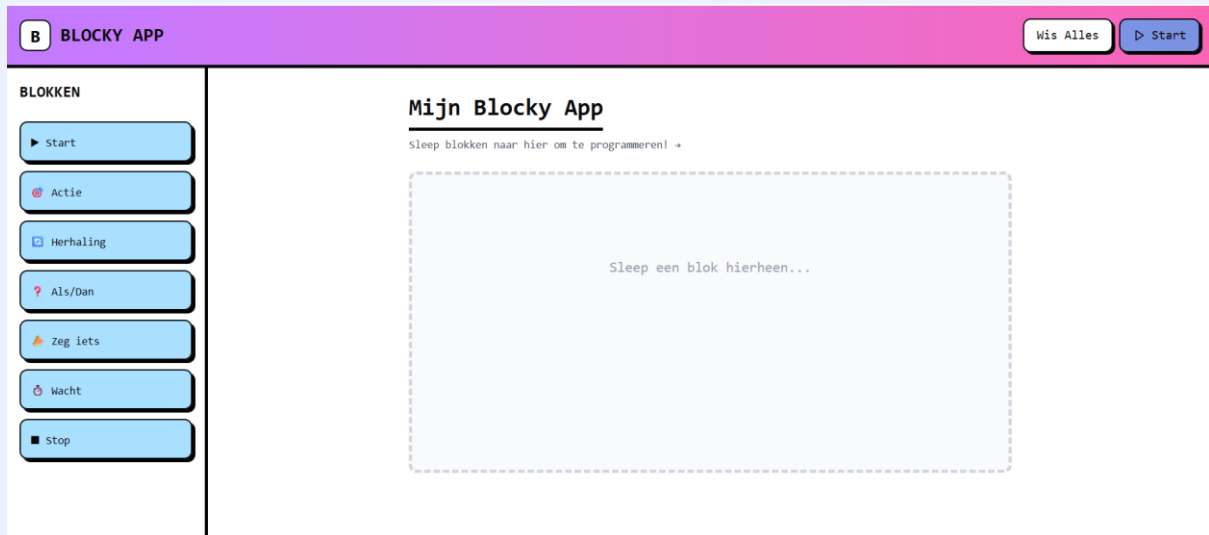
Figuur 3

Use Case Diagram van de functionele interacties binnen de test-automation tool.

Bron: Object Management Group (2017)

6.4 High-fi schetsen

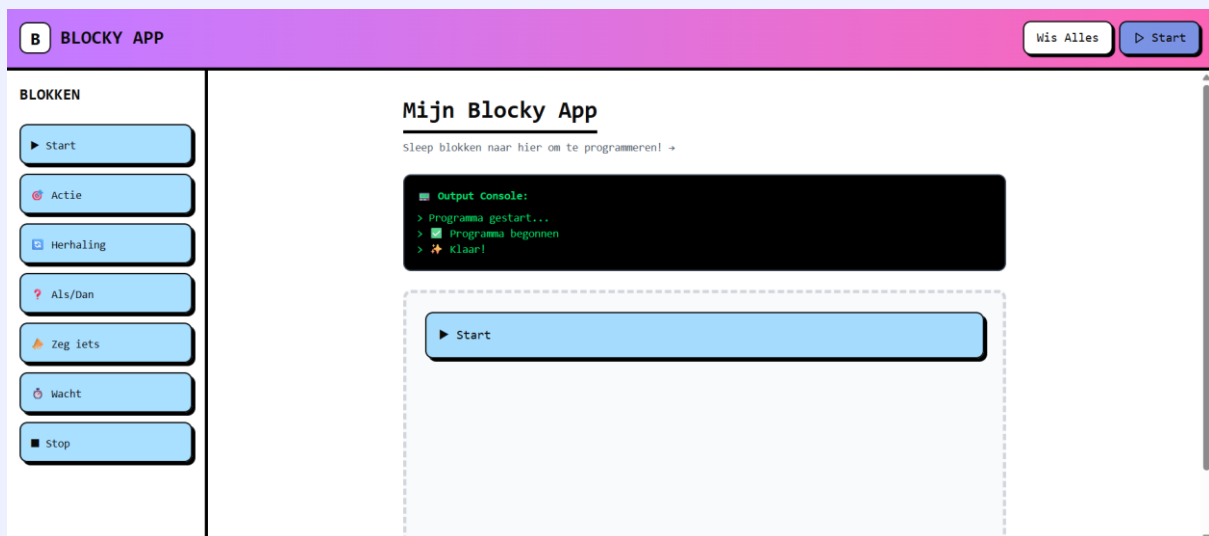
De High-fidelity schetsen (Figuur 4 en 5) tonen het visuele ontwerp van de Blockly App. De ontwerpen laten de interface zien waarbij gebruikers via een zijbalk verschillende logische blokken, zoals 'Start', 'Actie' en 'Wacht', naar het werkveld kunnen slepen. Figuur 5 toont daarnaast de Output Console, waarin de status van het uitgevoerde programma (zoals "Programma gestart") direct zichtbaar is voor de gebruiker.



Figuur 4

De basisinterface van de Blockly App met het zijmenu voor blokken en het lege werkveld.

Bron: Eigen uitwerking (2026).



Figuur 5

Interface-weergave van een geplaatst 'Start'-blok inclusief de Output Console met statusmeldingen.

Bron: Eigen uitwerking (2026).

6.5 Projectgrenzen (Scope)

- **Binnen scope:**

Het bouwen van de basis-editor (MVP), het automatisch maken van testcode en een werkende demo geven.

- **Buiten scope:**

Het beveiligen van de tool voor echt bedrijfsgebruik of het koppelen aan oude computersystemen van het bedrijf.

7. Kwaliteit en Begroting

Een betrouwbaar resultaat vraagt om controle. We gebruiken vaste regels om te bepalen wanneer werk echt 'af' is en schatten vooraf in hoeveel tijd taken kosten om vertraging te voorkomen.

7.1 Definition of Done (DoD)

Een taak is pas 'Done' als:

- De code of tekst is opgeslagen op de centrale plek (GitLab).
- Er een peer-review is uitgevoerd door een teamlid.
- De functionaliteit voldoet aan de acceptatiecriteria en de regressietest slaagt.

7.2 Story Points & Weging

Wij gebruiken de Fibonacci-reeks (1-8) om de omvang van taken in te schatten op basis van complexiteit, inspanning en onzekerheid.

Story Points	Complexiteit	1. Indicatie duur	2. Voorbeeld
1	Zeer klein	± 1–2 uur	aanpassing in code of documentatie, feedback verwerken
2	Klein	± halve dag	Bestaande user story verder uitwerken, feedback verwerken in meerdere onderdelen van een document
3	Middel	± 1 werkdag (licht)	Nieuwe kleine feature, retrospective uitwerken
5	Groot	± 1 volle werkdag	Complexere functionaliteit, technisch onderzoek

8	Zeer groot	± 1,5 – 2 werkdagen (max)	Grote feature of uitgebreide analyse met meerdere onderdelen
---	------------	---------------------------	--

Tabel 4

Story Points en weging per type taak.

Bron: Gebaseerd op Cohn (2005) en Scrum.org.

8. Planning

Om het project overzichtelijk te houden, werken we in vijf sprints van drie weken. We gebruiken GitLab om alle taken (issues) bij te houden. Zo weet iedereen precies wie waaraan werkt en wat de status is.

8.1 De Sprintcyclus

Elke sprint van drie weken volgt dezelfde vaste route. Dit zorgt voor rust en duidelijke momenten waarop we feedback vragen aan de opdrachtgever.

Fase binnen sprint	Activiteit	Verantwoordelijk	Moment
Sprintplanning	Taken bepalen en verdelen	Hele team	Start sprint
Ontwikkeling	Uitwerken toegewezen issues	Individuele teamleden	Week 1–2
Tussentijdse check	Voortgang bespreken	Team	Halverwege sprint
Afronding	Testen en controleren	Team	Week 3
Sprintreview	Presentatie aan opdrachtgever	Team + opdrachtgever	Einde sprint
Retrospective	Interne evaluatie	Team	Na sprintreview

Tabel 5

Overzicht van de vaste sprintcyclus en activiteiten per drie weken.

Bron: Gebaseerd op Scrum.org (2024) en Atlassian (z.d.).

8.2 De Roadmap (Sprintdoelen)

Elke sprint heeft een eigen focus. Hieronder zie je wat we per sprint gaan opleveren voor BlocklyTest.

Sprint	Focus	Activiteiten
Sprint 1	Probleemverkenning	Analyse van het probleem, verzamelen van requirements en data-analyse
Sprint 2	Ontwerp	Ontwerpen van het analysemodel en visualisaties
Sprint 3	Prototype ontwikkeling	Ontwikkelen van een eerste prototype voor analyse en visualisatie
Sprint 4	Functionaliteit uitbreiden	Ontwikkelen van advies- en uitlegfunctionaliteit
Sprint 5	Testen en oplevering	Testen van de oplossing, optimaliseren en voorbereiden van de oplevering

Tabel 6

Overzicht van de sprintdoelen en kernactiviteiten gedurende het project.

Bron: Eigen uitwerking (2026), gebaseerd op Scrum-iteraties.

Door op deze manier te werken, krijgt de opdrachtgever elke drie weken iets nieuws te zien. Hierdoor kunnen we snel aanpassingen doen als de wensen veranderen.

9. Ondertekening

Na akkoord door de opdrachtgever wordt dit plan van aanpak definitief vastgesteld.

Akkoord opdrachtgever:

Naam:

Handtekening:

Datum: __/__/2026

10. Literatuurlijst:

Alles over ERD's. (z.d.). Lucidchart. <https://www.lucidchart.com/pages/nl/tutorial/erd>

Artyukhovsky, A. (2026, February 6). Zakelijke voordelen van testautomatisering die elke CEO zou moeten weten. Innnowise. <https://innnowise.com/nl/blog/benefits-of-automated-testing/>

Atlassian. (z.d.). Agile Coach: Scrum. Geraadpleegd op 20 maart 2026, van <https://www.atlassian.com/agile/scrum>

Figma. (z.d.). Blocky app sketch. Geraadpleegd op 20 maart 2026, van <https://www.figma.com/make/aRR1OGb0fcCHbKjv7r0OTz/Blocky-app-sketch?t=ixXdQmEOaoYdbliV-0>

IB&P. (2020, July 3). ISO 31000: Referentiekader voor risicomanagement. Informatiebeveiliging & Privacy. <https://ib-p.nl/download/iso-31000-referentiekader-voor-risicomanagement/>

Michael W. Cohn & Mountain Goat Software. (2000). Agile estimating and planning. <https://www.mountaingoatsoftware.com/uploads/presentations/Agile-Estimating-Planning-Agile-2005.pdf>

Ministerie van Algemene Zaken. (2026, February 23). Sustainable Development Goals (SDG's): 17 doelen voor een duurzamere wereld. Rijksoverheid. <https://www.rijksoverheid.nl/onderwerpen/vn/duurzame-ontwikkelingsdoelen>

Object Management Group. (2017). Unified Modeling Language (UML) specification (Version 2.5.1). Geraadpleegd op 20 maart 2026, van <https://www.omg.org/spec/UML/2.5.1/PDF>

ISO. (2018). ISO 31000. International Organization for Standardization. <https://www.iso.org/files/live/sites/isoorg/files/store/en/PUB100426.pdf>

United Nations Development Programme. (z.d.). Sustainable Development Goals. Geraadpleegd op 19 maart 2026, van <https://www.undp.org/sustainable-development-goals>